



Universidad Nacional Autónoma de México



Faculty of Engineering
Electrical Engineering Division

Course: Compilers **Group:** 5
Semester: 2026-2

Report: **Parser**

Professor:
M.C. René Adrián Dávila Pérez

Student IDs:

320060829
320317264
320206968
320025561
320266083

Delivery Date:
Sunday, May 28th, 2026

Contents

1	Introduction	2
2	Theorical Framework	3
3	Development	6
3.1	Implementation	6
3.2	Grammar	6
3.2.1	Terminal tokens	6
3.2.2	Operator precedence and associativity	7
3.2.3	Grammar productions	7
3.2.4	Justification of the productions	9
3.2.5	Parse tree	10
3.3	Syntax-Directed Translation (SDT)	11
3.3.1	SDT Rules – Program structure	11
3.3.2	SDT Rules – Declarations	12
3.3.3	SDT Rules – Assignment	13
3.3.4	SDT Rules – Arithmetic expressions	14
3.3.5	SDT Rules – Conditions	15
3.3.6	SDT Rules – Control structures	16
3.3.7	SDT Rules – Input/Output	16
3.3.8	SDT Rules – Unary statements	17
3.4	Semantic analysis	17
3.4.1	Symbol table	17
3.4.2	Implemented semantic rules	17
3.4.3	Type inference auxiliary functions	18
3.4.4	AST traversal process	18
3.5	Input/Output and execution	19
3.6	Tests performed	19
4	Results	21
5	Conclusions	21
	References	21



1 Introduction

This document contains the necessary documentation for the comprehension of the construction of a syntactic analyzer and a Syntax-Directed Translation (SDT), which extend the lexical analyzer developed in the previous phase of this project. The system processes source code written in a subset of the C language, performing syntactic analysis through a bottom-up LALR(1) parser and semantic validation by means of SDT rules applied over the resulting Abstract Syntax Tree (AST).

The lexical phase, previously implemented using the Python Pygments library, produces a token flow that serves as input to the syntactic phase. Since PLY.YACC the tool selected for the LALR(1) parser, it requires a specific lexer interface, an adapter module was developed in order to integrate the two components. The parser then constructs an AST that is later traversed by the semantic analyzer, which enforces the typing and scoping rules of the language.

We should say that, as in the previous phase, the use of the FLEX and BISON tools is not permitted. Consequently, PLY (Python Lex-Yacc) was selected as the parser generator, which imposes specific requirements on both the grammar structure and the lexer output format, leading to a significant redesign of the Context-Free Grammar defined in the previous phase.

Objectives

- Extend the lexical analyzer developed in the previous phase into a full syntactic and semantic analysis for a subset of the C language.
- Adapt the Context-Free Grammar defined in the previous phase to be compatible with PLY.YACC's LALR(1) bottom-up parsing strategy, converting right-recursive productions to left-recursive form and delegating operator precedence to PLY's precedence mechanism.
- Construct an Abstract Syntax Tree (AST) during syntactic analysis by associating semantic actions with each grammar production rule, following the Syntax-Directed Translation approach.
- Implement a symbol table with scope management to enforce semantic rules, including variable declaration before use, type compatibility in assignments, valid operands in expressions, and correct argument usage in printf and scanf.
- Provide both a terminal-based and a Streamlit graphical interface for code input, analysis execution, and result visualization, including an interactive AST graph view.



2 Theoretical Framework

Firstly, the construction of a syntactic and semantic analyzer requires a solid understanding of several theoretical concepts that form the foundation of compiler design. [1]

In this project we address the intermediate stages of the front-end of the compiler: syntactic analysis and semantic validation through Syntax-Directed Translation. These phases work together to verify that the source code provided by the user is both structurally and semantically correct before proceeding with any code generation. As in the previous phase, the validation process is applied to code written in C that uses a specific set of instructions.

2.1 Syntactic Analysis (Parsing) and LALR(1)

The syntactic analyzer is the second stage of the compiler. It receives the token flow generated by the lexical analyzer and verifies that the sequence in order to accomplish the structural rules of the language, as defined by a Context-Free Grammar (CFG). [1]

In the previous phase of this project, a Top-Down Recursive Descent Parser was implemented, which constructs the parse tree starting from the initial grammar symbol and expanding downward toward the tokens. This approach required right-recursive production rules and explicit operator precedence layering within the grammar .

The current phase replaces this approach with a Bottom-Up LALR(1) parser, as implemented by PLY.YACC. In bottom-up parsing, the parse tree is constructed from the leaves (tokens) upward to the root (start symbol) through a sequence of shift and reduce operations. LALR(1) (Look-Ahead LR with 1 token of lookahead) is a widely used variant, it combines similar parser states to make a smaller and more efficient parsing table, while still working for most real programming languages.

A big advantage is that it handles left-recursive rules (which cause problems in top-down parsers) without issues in fact, they help make parsing faster.

Also, PLY.YACC lets you directly define operator precedence and associativity, so you don't have to complicate your grammar to handle them.

2.2 Grammar Adaptation for LALR(1)

A Context-Free Grammar (CFG) is a way to describe how a language is built. It is defined by four parts: $G=(V,T,P,S)$, where V is the set of non-terminal symbols (things that can be expanded), T is the set of terminal symbols (actual tokens of the language), P is the set of production rules written like $A \rightarrow \alpha$, and S is the start symbol. [3]

In the previous phase, the grammar used right-recursive rules to avoid left recursion, since top-down parsers cannot handle left recursion. For the LALR(1) parser, these rules were changed to left-recursive form, as recommended by PLY. For example:

$$\text{stmt_list} \rightarrow \text{stmt} \mid \text{stmt stmt_list}$$

(right-recursive, previous phase)


$$\text{stmt_list} \rightarrow \text{stmt_list stmt} \mid \text{stmt}$$

(left-recursive, current phase)

Also, operator precedence (like for arithmetic and logical operations) was removed from the grammar rules and instead defined using PLY's precedence tuple. This lets you specify priority and associativity (left, right, or none) in a simpler way, making the grammar cleaner. Finally, epsilon (empty) productions were used as little as possible. When they were necessary—such as in optional parts of a `for`-loop—an explicit empty rule was created using a non-terminal called `empty`, since PLY.YACC does not have a special way to write epsilon directly. [2]

2.3 Abstract Syntax Tree (AST)

The main goal of the parser is to take a sequence of tokens and organize them into a tree structure called an Abstract Syntax Tree (AST). Unlike a full parse tree, the AST removes unnecessary details like parentheses and punctuation, keeping only the important parts that give meaning to the program. This makes it easier to understand how the code is structured and to check that it follows the correct syntax. [1]

In this project, each node of the AST contains four things: the *kind* (the type of construct, like `"assign_stmt"` or `"if_stmt"`), the *children* (a list of child nodes or values), the *lineno* (the line number in the source code, useful for errors), and the *value* (the actual text for terminal symbols). The AST is built step by step during parsing, where each grammar rule creates its corresponding node using a function called `make_node`. [6]

2.4 Syntax-Directed Translation (SDT)

Syntax-Directed Translation (SDT) is a method where grammar rules are linked with actions that give them meaning. For each rule like $A \rightarrow \alpha$, some actions are executed when the parser uses that rule. These actions usually compute values (called synthesized attributes) that move from the child nodes up to the parent node. [1]

In this project, SDT is used in two steps. First, during parsing, each grammar rule creates an AST node as its result. Then, after parsing, a separate semantic analyzer goes through the AST and checks meanings by applying rules to each node. This two-step process separates building the structure from checking its correctness.

During semantic analysis, a detailed SDT trace is kept. For each node visited, it records the step number, the type of node, the line number, the action performed, and the state of the symbol table at that moment. This makes it easier to see what the program is doing and helps with debugging. [5]

2.5 Symbol Table and Scope Management

A symbol table is a data structure used during compilation to store information about variables (identifiers) in the program. At least, it keeps the name of each variable and its type.



It is used during semantic analysis to check that variables are declared before being used and that their types are used correctly. [1]

Scope management is handled using a stack of dictionaries, where each dictionary represents a different block of code. When the parser enters a new block (like inside an `if`, `while`, or `for`), a new scope is added to the stack. When leaving the block, that scope is removed. This makes sure that variables only exist inside the block where they were declared. When searching for a variable, the system looks from the most recent scope to the oldest one, following the usual scoping rules of C.[8]

2.6 Semantic Validation Rules

Semantic analysis checks that a program makes sense, even if it is written with correct syntax. In this project, the following simple rules are applied:

- **Declaration before use:** A variable must be declared before it is used. If not, an “undeclared variable” error is reported with the line number.
- **Type compatibility in assignment:** The value assigned to a variable must match its type. Types follow this order for compatibility: `char` $\leq$ `int` $\leq$ `float` $\leq$ `double`.
- **Valid operands in expressions:** Values used in expressions must be valid types like numbers or characters. If a variable is not declared, it causes an error that affects the whole expression.
- **printf and scanf argument validation:** The number and types of arguments must match the format specifiers. For example, `%d` is for `int`, `%f` for `float`/`double`, `%c` for `char`, and `%s` for strings. If they don’t match, an error is generated.
- **Evaluable conditions:** Conditions in `if`, `while`, and `for` must be of a type that can be evaluated as true or false, such as `bool`, `int`, `float`, `double`, or `char`.

2.7 Lexer Adapter and PLY Compatibility

PLY.YACC expects the lexer to have a `token()` method that returns one token at a time, where each token has `.type` and `.value`. When there are no more tokens, it should return `None`. However, the lexer used in this project (based on Pygments) gives a list of `(type, value)` pairs, which is not directly compatible.[7]

To fix this, an Adapter pattern was used. The class `PlyLexerAdapter` takes the list of tokens, converts the type names to the uppercase format expected by PLY, and provides a `token()` method that returns tokens one by one.

This way, the difference between the lexer and the parser is handled in one place, so both parts can work correctly without needing to change their original design. [4]



3 Development

3.1 Implementation

The compiler followed an architecture divided into three layers, where first we have the logic layer, then we have the presentation layer which is the frontend and finally we have the utilities layer, for this the logic layer is composed of six sections, the first one is the lexical analyzer, the token adapter, the grammatical rules, the AST structures, the semantic validation rules and the main phase that integrates the complete logic of syntactic and semantic analysis.

The flow follows a pipeline in a sequential way, where first, the lexical analyzer tokenizes the C source code using Pygments as a base. After this, a secondary block transforms the lexer output to the format required by PLY (Python Lex-Yacc) and then, the PLY LALR(1) parser analyzes the tokens and builds the AST. Finally, the semantic module traverses the AST applying the defined validation rules.

3.2 Grammar

This grammar describes a subset of the C language, supporting variable declarations, assignments, control structures (`if`, `while`, `for`), arithmetic, logical and relational operations, as well as input/output instructions (`printf`, `scanf`). The built parser uses PLY to generate a LALR(1) analyzer from these productions.

3.2.1 Terminal tokens

The grammar is based on the following sets of terminal tokens and these are generated by the lexical analyzer and transformed through an adapter to be compatible with PLY:

Table 1: Terminal tokens of the grammar

Category	Tokens
Data types	INT, FLOAT, DOUBLE, CHAR
Reserved words	IF, WHILE, FOR, PRINTF, SCANF
Identifiers and values	IDENTIFIER, VALUE_NUM, VALUE_CHAR, TEXT_STRING
Assignment operators	ASSIGN (=), ADD_ASSIGN (+=), SUB_ASSIGN (-=), DIV_ASSIGN (/=)
Arithmetic operators	PLUS, MINUS, TIMES, DIVIDE, MOD
Relational operators	EQ, NE, LT, GT, LE, GE
Logical operators	AND, OR, NOT
Unary operators	INC (++), DEC (--)
Punctuation	SEMICOLON, COMMA, LPAREN, RPAREN, LBRACE, RBRACE
Special	AMPERSAND (&)



3.2.2 Operator precedence and associativity

To resolve ambiguities in the grammar, the following precedence rules were defined from lowest to highest priority:

Table 2: Operator precedence rules

Priority	Associativity	Operators
1 (lowest)	Left	OR ()
2	Left	AND (&&)
3	Non-associative	EQ, NE, LT, GT, LE, GE
4	Left	PLUS, MINUS
5	Left	TIMES, DIVIDE, MOD
6 (highest)	Right	NOT (!)

3.2.3 Grammar productions

Going back to the original grammar we have the following:

Table 3: Productions of the context-free grammar

R	Non-terminal	Production
1	<i>program</i>	$\rightarrow$ <i>stmt_list</i>
2	<i>stmt_list</i>	$\rightarrow$ <i>stmt_list</i> <i>stmt</i>
3		<i>stmt</i>
4	<i>stmt</i>	$\rightarrow$ <i>decl_num</i>
5		<i>decl_char</i>
6		<i>assign_stmt</i>
7		<i>if_stmt</i>
8		<i>while_stmt</i>
9		<i>for_stmt</i>
10		<i>print_stmt</i>
11		<i>scan_stmt</i>
12		<i>unary_stmt</i>
13	<i>decl_num</i>	$\rightarrow$ <i>type_num</i> IDENTIFIER ;
14		<i>type_num</i> IDENTIFIER = <i>math_expr</i> ;
15	<i>decl_char</i>	$\rightarrow$ <i>type_char</i> IDENTIFIER ;
16		<i>type_char</i> IDENTIFIER = VALUE_CHAR ;
17	<i>type_num</i>	$\rightarrow$ int float double
18	<i>type_char</i>	$\rightarrow$ char
19	<i>assign_stmt</i>	$\rightarrow$ IDENTIFIER <i>assign_expr_op</i> <i>math_expr</i> ;



R	Non-terminal	Production
20		IDENTIFIER = VALUE_CHAR ;
21	<i>assign_expr_op</i>	→ = += -= /=
22	<i>if_stmt</i>	→ if (<i>cond</i>) { <i>stmt_list</i> }
23	<i>while_stmt</i>	→ while (<i>cond</i>) { <i>stmt_list</i> }
24	<i>for_stmt</i>	→ for (<i>for_init</i> ; <i>for_cond</i> ; <i>for_step</i>) { <i>stmt_list</i> }
25	<i>for_init</i>	→ <i>type_num</i> IDENTIFIER
26		<i>type_num</i> IDENTIFIER = <i>math_expr</i>
27		IDENTIFIER <i>assign_expr_op</i> <i>math_expr</i>
28		ϵ
29	<i>for_cond</i>	→ <i>cond</i>
30		ϵ
31	<i>for_step</i>	→ <i>inc_dec_op</i> IDENTIFIER
32		IDENTIFIER <i>inc_dec_op</i>
33		IDENTIFIER <i>assign_expr_op</i> <i>math_expr</i>
34		ϵ
35	<i>print_stmt</i>	→ printf (<i>print_args</i>) ;
36	<i>print_args</i>	→ TEXT_STRING
37		TEXT_STRING , <i>id_list</i>
38	<i>scan_stmt</i>	→ scanf (TEXT_STRING , & IDENTIFIER) ;
39	<i>id_list</i>	→ <i>id_list</i> , IDENTIFIER
40		IDENTIFIER
41	<i>unary_stmt</i>	→ <i>inc_dec_op</i> IDENTIFIER ;
42		IDENTIFIER <i>inc_dec_op</i> ;
43	<i>inc_dec_op</i>	→ ++ --
44	<i>math_expr</i>	→ <i>math_expr</i> + <i>math_expr</i>
45		<i>math_expr</i> - <i>math_expr</i>
46		<i>math_expr</i> * <i>math_expr</i>
47		<i>math_expr</i> / <i>math_expr</i>
48		<i>math_expr</i> % <i>math_expr</i>
49		IDENTIFIER
50		VALUE_NUM
51		(<i>math_expr</i>)
52	<i>cond</i>	→ <i>cond</i> <i>cond</i>
53		<i>cond</i> && <i>cond</i>
54		<i>math_expr</i> <i>rel_op</i> <i>math_expr</i>



R	Non-terminal	Production
55		! <i>cond</i>
56		(<i>cond</i>)
57		<i>math_expr</i>
58		VALUE_CHAR
59	<i>rel_op</i>	→ == != < > <= >=
60	<i>empty</i>	→ ε

3.2.4 Justification of the productions

The grammar is composed of 25 non-terminals and 60 productions that define the language:

Program structure: Rule 1 defines that a program consists of a list of statements, which is built recursively from the left using Rules 2 and 3.

Statements: Rules 4 through 12 establish that each statement can be one of nine alternatives: numeric declaration, character declaration, assignment, if conditional, while loop, for loop, printing with `printf`, reading with `scanf`.

Declarations: Rules 13 through 14 define numeric variable declarations (`int`, `float`, `double`), both simple and with initialization. Rules 15 and 16 do the same but for the character type (Rule 18), where initialization only accepts character literals.

Assignments: Rule 19 defines assignment with simple or compound operators (Rule 21: `=`, `+=`, `-=`, `/=`) over the arithmetic part and Rule 20 allows direct assignment of characters.

Control structures: Rules 22 and 23 define control blocks, such as `if` and `while`, which require a condition in parentheses and a block that starts and ends with curly braces. Rule 24 defines the `for` loop with three parts separated by semicolons: initialization (Rules 25–28), condition (Rules 29–30) and step (Rules 31–34), each of which can be empty (ε).

Input/Output: Rule 35 indicates the `printf` instruction, whose arguments are specified in Rules 36–37. Rule 38 indicates `scanf`, which accepts a format string and an identifier that must be followed by `&`.

Arithmetic expressions: Rules 44 through 51 indicate mathematical expressions with five operators, identifiers, numeric constants and grouping with parentheses.

Conditions: Rules 52 through 58 define conditions with logical operators, negation, relational comparisons (Rule 59), grouping with parentheses and the conversion of arithmetic expressions into booleans.

3.2.5 Parse tree

To give an example of how the LALR(1) works, the parse tree generated for the following input is used:

```
x = x + 1;
```

The LALR(1) parser implemented with PLY performs shift operations of tokens onto the stack and reduces when it recognizes the right side of a production at the top of the stack.

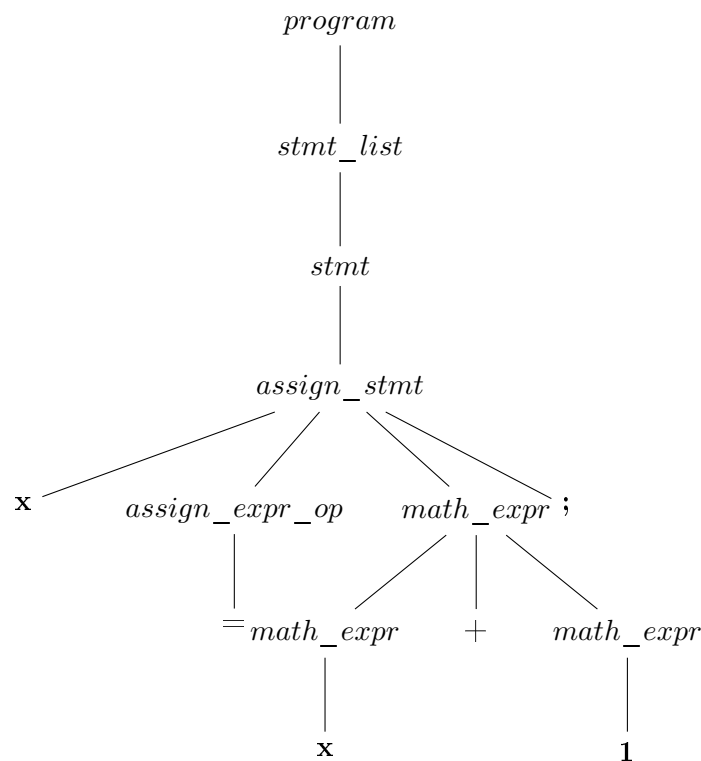


Figure 1: Parse tree for the input `x = x + 1;`

The bottom-up process for this input works as follows, where first the analyzer shifts the token `x` onto the stack, then shifts the token `=` and reduces it to the non-terminal using Rule 21. Then it shifts the second token `x` and reduces it to *math_expr* thru Rule 49. Then it shifts `+` and the token `1`, which is reduced to *math_expr* by Rule 50.

At this point, the parser has at the top of the stack the following elements:

math_expr + *math_expr*

Which is the same as the right-hand side of Rule 44.



With this said, this reduction is applied which generates a *math_expr* node that represents the expression $x + 1$. Finally the token `;` is shifted and the stack contains the elements

IDENTIFIER *assign_expr_op math_expr* ;

And this is the same as with Rule 19 and this reduction is applied creating the *assign_stmt* node, which is reduced to *stmt* by Rule 6, then to *stmt_list* by Rule 3, and finally to the initial symbol *program* thru Rule 1.

3.3 Syntax-Directed Translation (SDT)

The Syntax-Directed Translation (SDT *Syntax-Directed Translation*) is that it links the semantic rules to each production of the grammar.

For this, with the working approach already established, we can implement and present the SDT rules for each production, where:

SymTable represents the symbol table.

.type is the inferred type attribute.

.node is the generated AST node.

3.3.1 SDT Rules – Program structure

$$program \rightarrow stmt_list \quad \{program.node = make_node("program", stmt_list.nodes)\}$$

$$stmt_list \rightarrow stmt_list_1 stmt \quad \{stmt_list.nodes = stmt_list_1.nodes \cup \{stmt.node\}\}$$

$$stmt_list \rightarrow stmt \quad \{stmt_list.nodes = \{stmt.node\}\}$$

$$\begin{aligned} stmt \rightarrow & decl_num \mid decl_char \mid assign_stmt \mid if_stmt \\ & \mid while_stmt \mid for_stmt \mid print_stmt \mid scan_stmt \\ & \mid unary_stmt \quad \{stmt.node = X.node\} \end{aligned}$$



3.3.2 SDT Rules – Declarations

```
decl_num → type_num id ;
          {if SymTable.declare(id.name, type_num.type) then ok
           else error("variable redeclaration")}
```

```
decl_num → type_num id = math_expr ;
          {if SymTable.declare(id.name, type_num.type)
           then if compatible(type_num.type, math_expr.type) then ok
           else error("incompatible type in assignment")
           else error("variable redeclaration")}
```

```
decl_char → type_char id ;
           {if SymTable.declare(id.name, "char") then ok
            else error("redeclaration")}
```

```
decl_char → type_char id = VALUE_CHAR ;
           {if SymTable.declare(id.name, "char") then ok
            else error("redeclaration")}
```

```
type_num → int   {type_num.type = "int"}
type_num → float {type_num.type = "float"}
type_num → double {type_num.type = "double"}
type_char → char {type_char.type = "char"}
```



3.3.3 SDT Rules – Assignment

$assign_stmt \rightarrow id \ assign_op \ math_expr ;$
 {if $SymTable.lookup(id.name) = null$
 then error("variable not declared")
 else if $assign_op.op = "="$
 then if $compatible(SymTable.lookup(id.name), math_expr.type)$
 then ok else error("incompatible type")
 else if $assign_op.op \in \{ "+=", "-=", "/=" \}$
 then if $numeric(SymTable.lookup(id.name)) \wedge numeric(math_expr.type)$
 then ok else error("non-numeric operands")}

$assign_stmt \rightarrow id = VALUE_CHAR ;$
 {if $SymTable.lookup(id.name) = null$
 then error("variable not declared")
 else if $SymTable.lookup(id.name) = "char"$ **then ok**
 else error("incompatible type")}

$assign_op \rightarrow = \{ assign_op.op = "=" \}$
 | $+ = \{ assign_op.op = "+=" \}$
 | $- = \{ assign_op.op = "-=" \}$
 | $/ = \{ assign_op.op = "/=" \}$



3.3.4 SDT Rules – Arithmetic expressions

$math_expr \rightarrow math_expr_1 + math_expr_2$
 {if numeric($math_expr_1.type$) $\wedge$ numeric($math_expr_2.type$)
 then $math_expr.type = \text{promote}(math_expr_1.type, math_expr_2.type)$
 else error(“non-numeric operands”)}

$math_expr \rightarrow math_expr_1 - math_expr_2$
 { $math_expr.type = \text{promote}(math_expr_1.type, math_expr_2.type)$ (same rule)}

$math_expr \rightarrow math_expr_1 * math_expr_2$
 { $math_expr.type = \text{promote}(math_expr_1.type, math_expr_2.type)$ (same rule)}

$math_expr \rightarrow math_expr_1 / math_expr_2$
 { $math_expr.type = \text{promote}(math_expr_1.type, math_expr_2.type)$ (same rule)}

$math_expr \rightarrow math_expr_1 \% math_expr_2$
 { $math_expr.type = \text{promote}(math_expr_1.type, math_expr_2.type)$ (same rule)}

$math_expr \rightarrow \text{id}$ {if $SymTable.lookup(id.name) \neq \text{null}$
 then $math_expr.type = SymTable.lookup(id.name)$
 else error(“variable not declared”)}

$math_expr \rightarrow \text{VALUE_NUM}$ { $math_expr.type = \text{infer}(\text{VALUE_NUM.value})$ }

$math_expr \rightarrow (math_expr_1)$ { $math_expr.type = math_expr_1.type$ }

3.3.5 SDT Rules – Conditions

$$\begin{aligned} cond &\rightarrow cond_1 \parallel cond_2 \\ &\quad \{\text{if } logical(cond_1.type) \wedge logical(cond_2.type) \\ &\quad \quad \text{then } cond.type = \text{"bool"} \\ &\quad \quad \text{else error("invalid operands for logical OR")}\} \\ \\ cond &\rightarrow cond_1 \&\& cond_2 \\ &\quad \{\text{if } logical(cond_1.type) \wedge logical(cond_2.type) \\ &\quad \quad \text{then } cond.type = \text{"bool"} \\ &\quad \quad \text{else error("invalid operands for logical AND")}\} \\ \\ cond &\rightarrow math_expr_1 _rel_op _math_expr_2 \\ &\quad \{\text{if } numeric(math_expr_1.type) \wedge numeric(math_expr_2.type) \\ &\quad \quad \text{then } cond.type = \text{"bool"} \\ &\quad \quad \text{else error("non-numeric operands in comparison")}\} \\ \\ cond &\rightarrow ! \ cond_1 \\ &\quad \{\text{if } logical(cond_1.type) \text{ then } cond.type = \text{"bool"} \\ &\quad \quad \text{else error("invalid operand for negation")}\} \\ \\ cond &\rightarrow (\ cond_1 \) \quad \{cond.type = cond_1.type\} \\ \\ cond &\rightarrow math_expr \quad \{cond.type = math_expr.type\} \\ \\ cond &\rightarrow \text{VALUE_CHAR} \quad \{cond.type = \text{"char"}\} \end{aligned}$$

Where the $logical(t)$ function returns true if $t \in \{\text{"bool"}, \text{"int"}, \text{"float"}, \text{"double"}, \text{"char"}\}$, these are the data types that are evaluated as a logical function.

$$\begin{aligned} rel_op &\rightarrow == \{rel_op.op = \text{"=="}\} \mid != \{rel_op.op = \text{"!="}\} \\ &\mid < \{rel_op.op = \text{"<"}\} \mid > \{rel_op.op = \text{">"}\} \\ &\mid <= \{rel_op.op = \text{"<="}\} \mid >= \{rel_op.op = \text{">="}\} \end{aligned}$$



3.3.6 SDT Rules – Control structures

$$\begin{aligned}
 if_stmt &\rightarrow \text{if } (cond) \{ stmt_list \} \\
 &\quad \{ \text{if logical}(cond.type) \text{ then ok} \\
 &\quad \text{else error("condition not evaluable in if")} \} \\
 while_stmt &\rightarrow \text{while } (cond) \{ stmt_list \} \\
 &\quad \{ \text{if logical}(cond.type) \text{ then ok} \\
 &\quad \text{else error("condition not evaluable in while")} \} \\
 for_stmt &\rightarrow \text{for } (for_init ; for_cond ; for_step) \{ stmt_list \} \\
 &\quad \{ SymTable.enter_scope(); \\
 &\quad \text{evaluate}(for_init); \\
 &\quad \text{if } for_cond \neq \varepsilon \wedge \neg \text{logical}(for_cond.type) \\
 &\quad \quad \text{then error("condition not evaluable in for");} \\
 &\quad \text{evaluate}(for_step); \text{evaluate}(stmt_list); \\
 &\quad SymTable.exit_scope() \}
 \end{aligned}$$

3.3.7 SDT Rules – Input/Output

$$\begin{aligned}
 print_stmt &\rightarrow \text{printf } (print_args) ; \\
 &\quad \{ \text{let } specs = \text{extract_specs}(\text{TEXT_STRING.value}) \\
 &\quad \text{if } |specs| \neq |id_list| \text{ then error("incorrect number of arguments")} \\
 &\quad \text{else } \forall (spec_i, id_i) \in \text{zip}(specs, id_list) : \\
 &\quad \quad \text{if } SymTable.lookup(id_i) = \text{null} \\
 &\quad \quad \text{then error("variable not declared")} \\
 &\quad \quad \text{else if } \neg \text{format_ok}(SymTable.lookup(id_i), spec_i) \\
 &\quad \quad \text{then error("incompatible format")} \} \\
 scan_stmt &\rightarrow \text{scanf } (\text{TEXT_STRING}, \&id) ; \\
 &\quad \{ \text{if } SymTable.lookup(id.name) = \text{null} \\
 &\quad \quad \text{then error("variable not declared")} \\
 &\quad \text{else let } specs = \text{extract_specs}(\text{TEXT_STRING.value}) \\
 &\quad \text{if } |specs| \neq 1 \text{ then error("exactly one specifier is expected")} \\
 &\quad \text{else if } \neg \text{format_ok}(SymTable.lookup(id.name), specs[0]) \\
 &\quad \quad \text{then error("format incompatible with variable type")} \}
 \end{aligned}$$



Where the `format_ok` function verifies that a variable type and a format specifier are compatible: `%d/%i` needs an `int` variable, `%f` needs a `float` variable, to put them as an example.

3.3.8 SDT Rules – Unary statements

```

unary_stmt → inc_dec_op id ;
              {if SymTable.lookup(id.name) = null
               then error("variable not declared")
               else if ¬numeric(SymTable.lookup(id.name))
               then error("unary operator requires numeric type")}

unary_stmt → id inc_dec_op ; (same semantic rule)

inc_dec_op → ++ {inc_dec_op.op = "++"}
            | -- {inc_dec_op.op = "--"}

```

3.4 Semantic analysis

The semantic analysis is the phase that comes after the syntactic one, where it receives the generated AST and traverses it recursively to apply the validation rules. the fundamental part is the symbol table and some auxiliary functions that allow inferring types, verifying compatibility and validating the language constructs.

3.4.1 Symbol table

The symbol table is implemented as a stack of dictionaries, where each dictionary represents a (*scope*) of variables. When this part enters a code block such as the block of an `if`, `while` or `for`, a new scope is created by pushing an empty dictionary. When leaving the block, the top scope is removed, then the main table operations are: *declare*, which registers a variable verifying that no redeclaration exists, *lookup*, which searches for a variable traversing the scopes from the innermost to the outermost, and *snapshot*, which gives us a combined view of all variables visible at the current point of analysis, this for the compiler's error messages.

3.4.2 Implemented semantic rules

The semantic rules that the compiler validates are:

- First, that every variable be declared before being used in any expression or input/output instruction.
- Second, type compatibility in assignments where assigning an `int` to a `float` is valid, but not the other way around.



- Third, that arithmetic operations require numeric operands and logical operations require types evaluable as booleans.
- Fourth, the validation of `printf` and `scanf` instructions, where it is checked that the number of format specifiers matches (`%d` with `int`, `%f` with `float/double`, `%c` with `char`).
- Fifth, that the conditions used in `if`, `while` and `for` structures are expressions that evaluate as boolean values of true or false.

3.4.3 Type inference auxiliary functions

The semantic part uses auxiliary functions for type inference and verification. The literal type inference function determines if a value is of character type, it does this by looking at its beginning and end, seeing if it has single quotes or string if it is delimited by double quotes. The constant type inference function determines if a number is integer or float according to the presence of a decimal point.

The numeric verification function validates if a type belongs to any of the numeric types such as (`int`, `float`, `double`, `char`).

The logical evaluability function verifies if a type can be used as a boolean condition, where it also accepts the `bool` type.

The assignment compatibility function implements the revision of which data types accept another data type: `double` accepts `float`, `int` and `char`; `float` accepts `int` and `char`; `int` accepts `char`.

The format specifier extraction function reviews the format strings of `printf/scanf` to identify the data types that are going to be both read and printed (`%d`, `%f`, `%c`, `%s`) ignoring the escape sequences.

3.4.4 AST traversal process

The semantic analysis is performed through two recursive functions that traverse the AST.

The first function determines the type of an expression by recursively evaluating the trees generated from the main tree, for identifiers, it queries the symbol table, for constants, it infers the type from the value, but for binary operations, it verifies that both operands are valid and applies type promotion.

The second function traverses each node of the AST applying the corresponding semantic rules according to the node type: for declarations, it registers the variable in the symbol table and verifies the compatibility of the initializer, but for assignments, it verifies that the variable exists and that the type is compatible, then for control structures, it checks that the condition is correct, now for the I/O instruction part, it verifies the format specifiers and the arguments.



3.5 Input/Output and execution

The program supports two execution modes: by command line (terminal mode) and through a graphical user interface implemented with Streamlit.

In terminal mode, the program presents an interactive menu where the user can choose between entering C code directly from the keyboard or loading a file from the input folder. The program processes the lexical, syntactic and semantic analysis in a sequential way, and shows a detailed report that includes the token table, tokens grouped by type, the result of the syntactic analysis, the errors found (if any) and the semantic analysis trace. In case a syntactic error is detected, the semantic analysis is not executed, since it is not possible to build a valid AST. The program allows running multiple analyses consecutively without restarting the application.

In graphical interface mode, the same functionality is offered but with a more visual and interactive presentation. Each phase of the analysis can be visualized in an organized way with colors and formats that facilitate reading. Additionally, the UI allows visualizing the graph of the generated AST, which supports zoom to examine large trees.

3.6 Tests performed

The following table summarizes all the compiler cases: the valid case, the cases with semantic errors and the cases with syntactic errors.

Table 4: Classification of errors detected by the parser and the SDT system.

Error type	Example	Description
Valid case. Integrated program	<pre>int x = 1; float y = 2; char c = 'a'; x = x + 1; y = y + x; if (x < y && y > 0) { x += 1; } printf("%d %f %c", x, y, c); scanf("%d", &x);</pre>	Declarations of three different types (Rules 14, 16), simple and compound assignments (Rule 19), arithmetic expressions with type promotion (Rule 44), conditional with logical and relational operators (Rules 22, 53, 54), and I/O with multiple format specifiers (Rules 35, 38). The symbol table registers x:int , y:float , c:char . Result: Syntax VALID, Semantics VALID .



Error type	Example	Description
Syntactic error. Incomplete declaration	<pre>int x = ;</pre>	The initialization expresion before the semicolon is missing. The parser reports: Syntax error near ';' (token SEMICOLON) at line 1. No AST is generated and the semantic analysis is not executed.
Syntactic error. Control structure without parentheses	<pre>int x = 1; if x > 0 { x = x + 1; }</pre>	The if statment omits the mandatory parentheses around the condition. The parser reports: Syntax error near 'x' (token IDENTIFIER) at line 2. Rule 22 explicitley requires the LPAREN and RPAREN tokens.
Semantic error. Undeclared variable	<pre>int x = 1; y = x + 2;</pre>	The variable y is used in an assignment without having been declared previosly. The syntax is valid acording to Rule 19, but when traversing the AST the symbol table does not find y. It is reported: Semantic error: variable 'y' not declared at line 2.
Semantic error. Incompatible type in assignment	<pre>char c = 'a'; c = 25;</pre>	An attempt is made to asign an integer value (int) to a variable of type char. The compatibility function rejects the assignment since char does not accept int in the promotion hierarchy. It is reported: Semantic error: cannot assign 'int' to 'char' at line 2.
Semantic error. Undeclared variable in condition	<pre>int x = 1; if (z > 0 && x) { x = x + 1; }</pre>	The if condition references the variable z that was not declared. The syntax is valid acording to Rule 22, but when evaluating the condition (Rule 54), the search for z in the symbol table fails. It is reported: Semantic error: variable 'z' not declared at line 2.



Error type	Example	Description
Semantic error. printf arguments	<pre>int x = 10; printf("%d %d", x);</pre>	The format string contains two specifiers (%d %d), but only one argument (x) is provided. The SDT rule of printf extracts the specifiers and verifies the corresponding match. It is reported: Semantic error: printf expected 2 args but got 1 at line 2.
Semantic error. In-compatible format in scanf	<pre>char c = 'a'; scanf("%d", &c);</pre>	The %d specifier requires a variable of type int, but c was declared as char. The format compatibility function verifies that %d is only compatible with int. It is reported: Semantic error: scanf format '%d' incompatible with 'char' at line 2.
Semantic error. Condition with undeclared variable	<pre>int x = 1; if (q > 0) { x = x + 1; }</pre>	The variable q does not exist in the symbol table. When evaluating the if condition, the search fails before being able to verify the relational operation. It is reported: Semantic error: variable 'q' not declared at line 2.

The nine test cases produce exactly the expected result. In the cases with syntactic error, the semantic analysis is not executed since it is not possible to build a valid AST. In all the semantic cases, the syntax is valid and the error is correctly detected by the SDT rules during the traversal of the AST, generating messages that include the type of error, the variable or token involved and the corresponding line number.

4 Results

5 Conclusions

References

- [1] A. Holub, *Compiler Design in C*. Prentice-Hall, 1990, ISBN: 0-13-155045-4. [Online]. Available: <https://holub.com/goodies/compiler/compilerDesignInC.pdf>



- [2] D. Beazley. “Ply (python lex-yacc) documentation.” version 3.11. [Online]. Available: <https://www.dabeaz.com/ply/ply.html>
- [3] GeeksforGeeks. “What is context-free grammar? ”[Online]. Available: <https://www.geeksforgeeks.org/theory-of-computation/what-is-context-free-grammar/>
- [4] GeeksforGeeks. “Lalr parser (look-ahead lr). ”[Online]. Available: <https://www.geeksforgeeks.org/compiler-design/lalr-parser-with-examples/>
- [5] GeeksforGeeks. “Syntax directed translation in compiler design. ”[Online]. Available: <https://www.geeksforgeeks.org/syntax-directed-translation-in-compiler-design/>
- [6] dev.to. “Abstract syntax tree (ast) deep dive: From theory to practical compiler implementation. ”[Online]. Available: https://dev.to/min_yi_e5fbf986e24f1c42df/abstract-syntax-tree-ast-deep-dive-from-theory-to-practical-compiler-implementation-4jpo
- [7] GeeksforGeeks. “Adapter design pattern. ”[Online]. Available: <https://www.geeksforgeeks.org/adapter-pattern/>
- [8] GeeksforGeeks. “Symbol table in compiler design. ”[Online]. Available: <https://www.geeksforgeeks.org/symbol-table-compiler/>
- [9] Pygments Contributors. “Builtin tokens,” Accessed: Mar. 13, 2026. [Online]. Available: <https://pygments.org/docs/tokens/>
- [10] GeeksforGeeks. “Ambiguous grammar. ”[Online]. Available: <https://www.geeksforgeeks.org/compiler-design/ambiguous-grammar/>
- [11] C. Wong. “Cfg developer. ”[Online]. Available: <https://web.stanford.edu/class/archive/cs/cs103/cs103.1156/tools/cfg/>